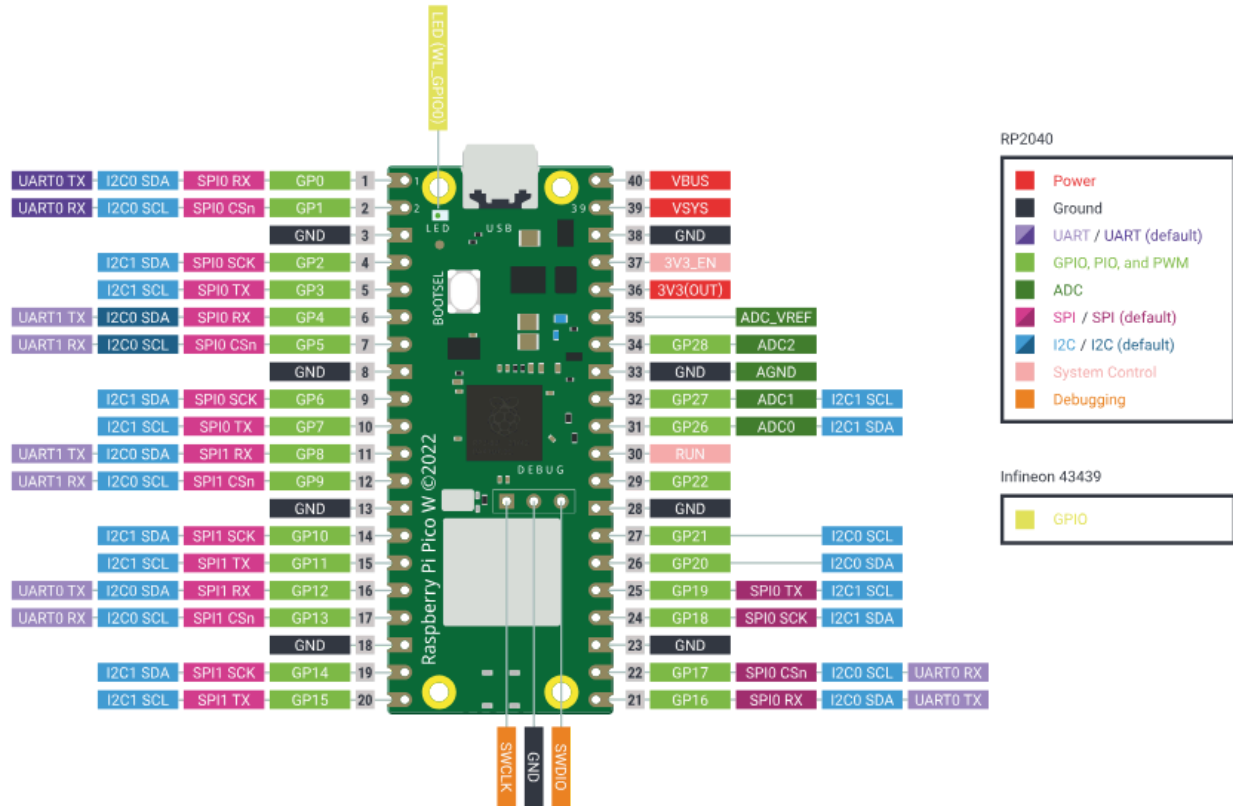


Traverse LED from Left To Right *on the Raspberry Pi Pico-W* using C SDK



GUIDED BY:
DURAI P S

MENTOR:
DANIEL RAJ A

TEAM:

1. KARTHICK A
2. LUCKSHANA V R
3. SHREE SUBI G
4. JANASRUTHIKA P R

TABLE OF CONTENT

S.no	Content	Page number
1.	Cloning sdk and examples to the directory	1
2.	Create project folder & export SDK path	4
3.	Build the project using CMake and Make	5
4.	Copy <code>.uf2</code> file to flash the microcontroller	6
5.	Appendix	8

Traverse LED from Left To Right *on the Raspberry Pi Pico-W using C SDK*

1. CLONING SDK AND EXAMPLES TO THE DIRECTORY:

1.1. To clone the SDK, Create a new directory and clone the SDK as given below

```
$ mkdir pico
$ cd pico
$ git clone -b master https://github.com/raspberrypi/pico-sdk.git
$ cd pico-sdk
$ git submodule update --init
```

1.2. To build applications we need CMake, ARM GNU Toolchain. To install those, run the command.

```
$ sudo apt update
$ sudo apt install cmake gcc-arm-none-eabi build-essential
libnewlib-arm-none-eabi minicom
```

- **Cmake** = CMake is an open-source, cross-platform tool used for managing the build process of software projects. It allows developers to define the build process in a platform-independent way by creating configuration files (CMakeLists.txt)
- **Gcc-arm-none-eabi** = Compiling C/C++ code for ARM Cortex-M microcontrollers.
Bare-metal development, meaning there's no OS — you directly program the hardware.
- **libnewlib-arm-none-eabi** = A C standard library (like stdio.h, math.h) optimized for embedded systems.

- **build-essential** = A meta-package that installs tools like make,g++ and other basic development tools needed to compile C/C++ code.

2. CREATE PROJECT FOLDER AND EXPORT SDK PATH:

2.1. When doing a new project, create a project folder:

```
$ cd pico
$ mkdir led_traverse
$ cd ed_traverse
```

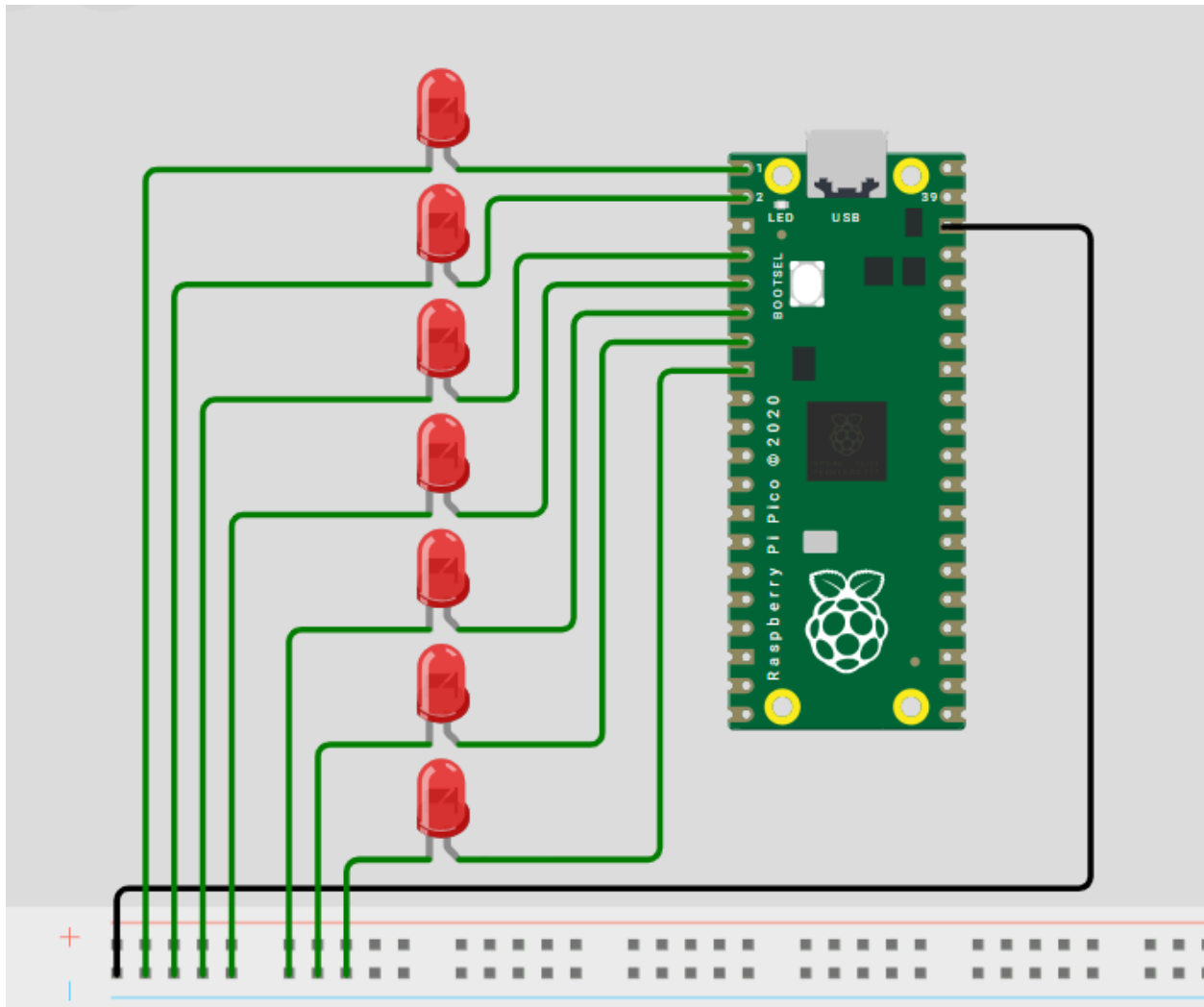
2.2. Create `led_traverse.c`

```
#include "pico/stdlib.h"

#define LED_PIN_BASE 0 // Starting GPIO pin (LEDs are connected
// from GPIO 0 to 7)
#define NUM_LEDS 8 // Total number of LEDs

int main() {
    stdio_init_all(); // Initialize standard input/output

    // Configure GPIO pins 0–7 as output for LEDs
    for (int i = 0; i < NUM_LEDS; i++) {
        gpio_init(LED_PIN_BASE + i); // Initialize each pin
        gpio_set_dir(LED_PIN_BASE + i, GPIO_OUT); // Set as output
    }
    while (true) {
        // Loop through LEDs from right (LED7) to left (LED0)
        for (int i = NUM_LEDS - 1; i >= 0; i--) {
            uint8_t mask = 1 << i; // Create bitmask for current LED
            gpio_clr_mask(0xFF); // Turn OFF all LEDs
            gpio_set_mask(mask); // Turn ON only the current LED
            sleep_ms(300); // Wait 300 ms for visibility
        }
    }
}
```



2.3. Create `CMakeLists.txt`:

```
cmake_minimum_required(VERSION 3.13)

include(pico_sdk_import.cmake)

project(led_traverse C CXX ASM)

pico_sdk_init()

add_executable(led_traverse
    led_traverse.c
)
```

```
target_link_libraries(led_traverse pico_stdlib)

pico_add_extra_outputs(led_traverse)
```

III. BUILD THE PROJECT USING CMAKE AND MAKE:

3.1. Compile the program.

```
$ cd ~/pico/led_input_project
$ mkdir build
$ cd build
$ cmake ..
$ make
```

- If the cmake file shows any error, update the CMake version after checking the version using.

```
cmake --version
```

```
$ sudo apt remove cmake
$ sudo snap install cmake --classic
```

IV. COPY .UF2 FILE TO FLASH THE MICROCONTROLLER:

4.1. After the completion of progress, copy the uf2 file.

```
$ cp led_traverse.uf2 /media/$USER/RPI-RP2/
```

Copy the led_input.uf2 file into the Pico folder (RPI-RP2).

```
$ cp led_traverse.uf2 /media/.../RPI-RP2
```

VII.APPENDIX:

REFERENCE LIST:

1. <https://www.digikey.in/en/maker/projects/raspberry-pi-pico-and-rp2040-cc-p-art-1-blink-and-vs-code/7102fb8bca95452e9df6150f39ae8422>
2. <https://youtu.be/JhajoAyP8e4?si=tR607pWw90qvuitR>
3. https://wellys.com/posts/rp2040_c_blink/